

Introduction to Blockchain and Smart Contracts

Lecture 3: Cryptography

Subodh Sharma | IIT Delhi | Jan 12, 2026





Recap

- Hash functions
- Merkle Trees
- Digital Signatures



Integrity: Cryptographic Hash Functions

- A hash function $H(\cdot)$ that has the following features:
 - Takes in **arbitrary-sized** input and produces a **fixed-size** output
 - It is a **non-invertible** function. That is, given an output one cannot compute the input
 - Forward computability is eminently **efficient** — typically $O(n)$ where n is the input size
- For hash functions to be secure following invariants must be satisfied:
 - **Collision-resistance:** It is infeasible to find two values, x, y , s.t. $x \neq y$ yet $H(x) = H(y)$ with high probability
 - **Hiding:** Given y s.t. $y = H(x)$, there is no feasible way to find x
 - In fact it is $H(r \parallel x)$ where r is chosen from a distribution with high min-entropy

Hash Functions: Puzzle Friendliness

- **Puzzle-friendliness:** if for every possible n -bit output value y , if k is chosen from a distribution with high min-entropy, then it is **infeasible** to find x such that $H(k || x) = y$ in time significantly less than 2^n .
 - Eg: $H(\text{BlkHeader} || k)$ has d leading zeros.
 - **No short-cut** to find the nonce k other than trying them exhaustively.
 - Probability of success in each try: $\frac{1}{2^d}$; $E[\text{tries}] \simeq 2^d$

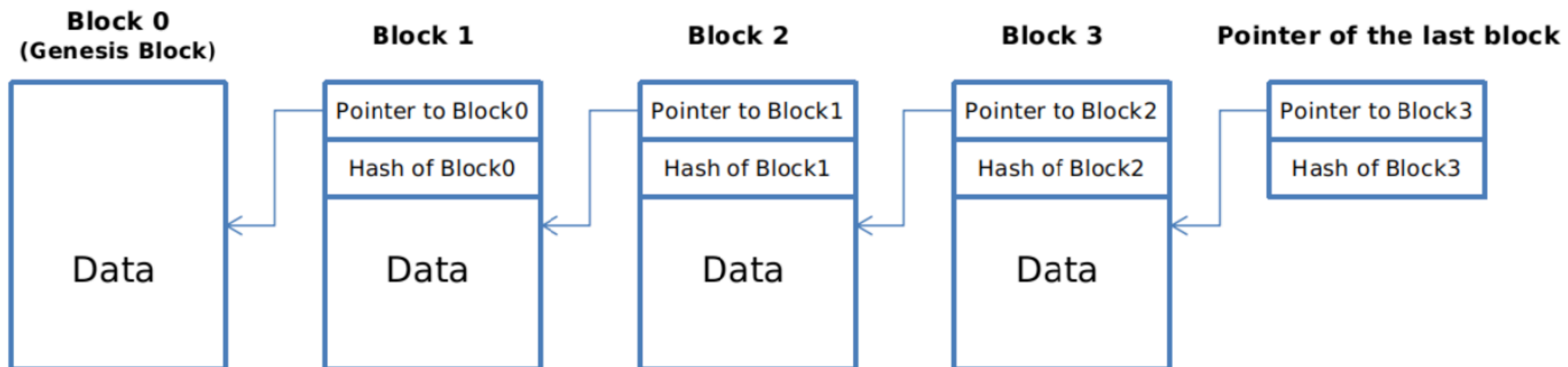
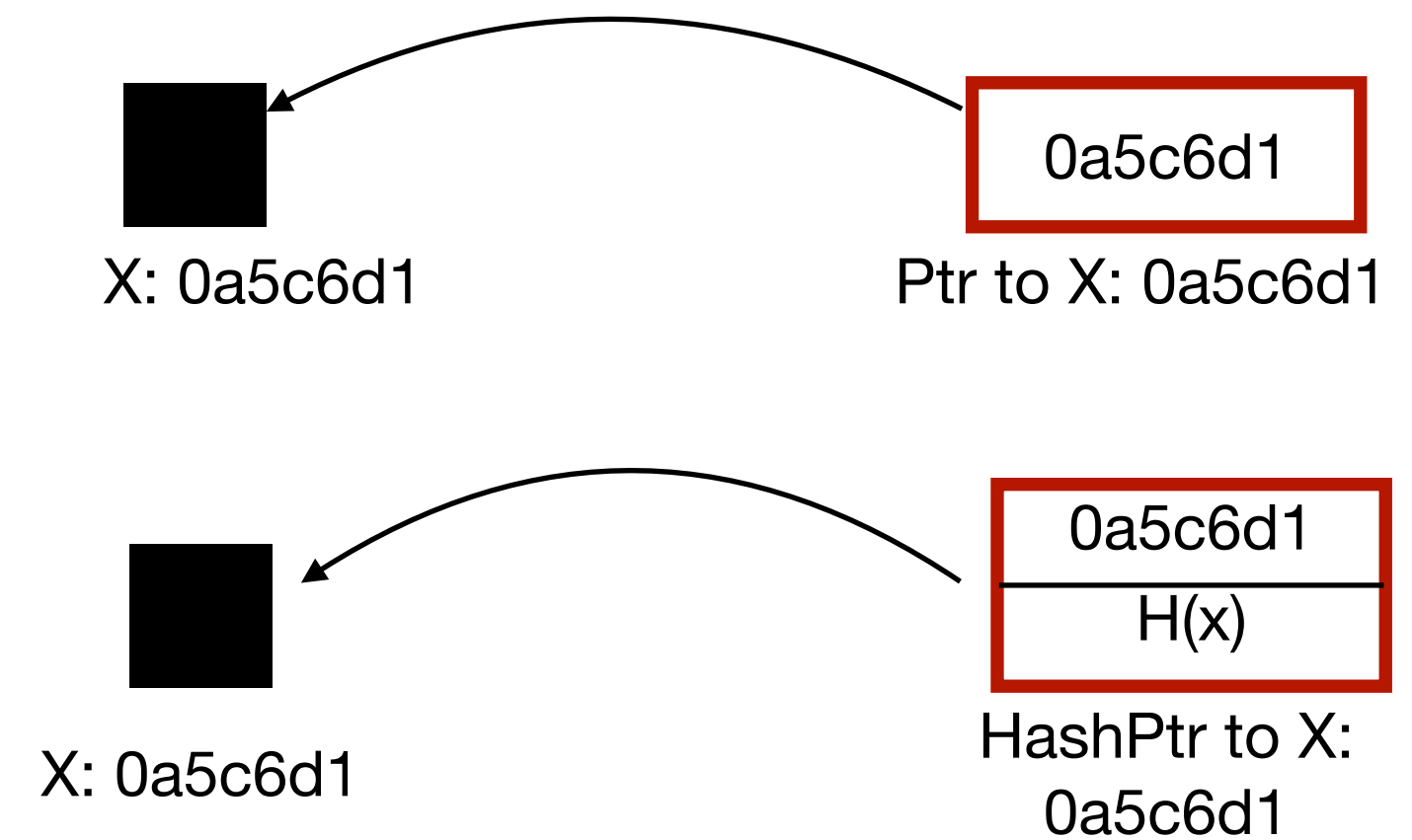
Hash Functions: Core Properties

- **Uniform Distribution, Low Collision Rate, Avalanche Effect**
- **Pre-image resistance & Second Pre-image resistance**
 - In addition to collision resistance (discussed a few slides back)

From Hashes to Hash Pointers

From Integrity to Tamper-evidence

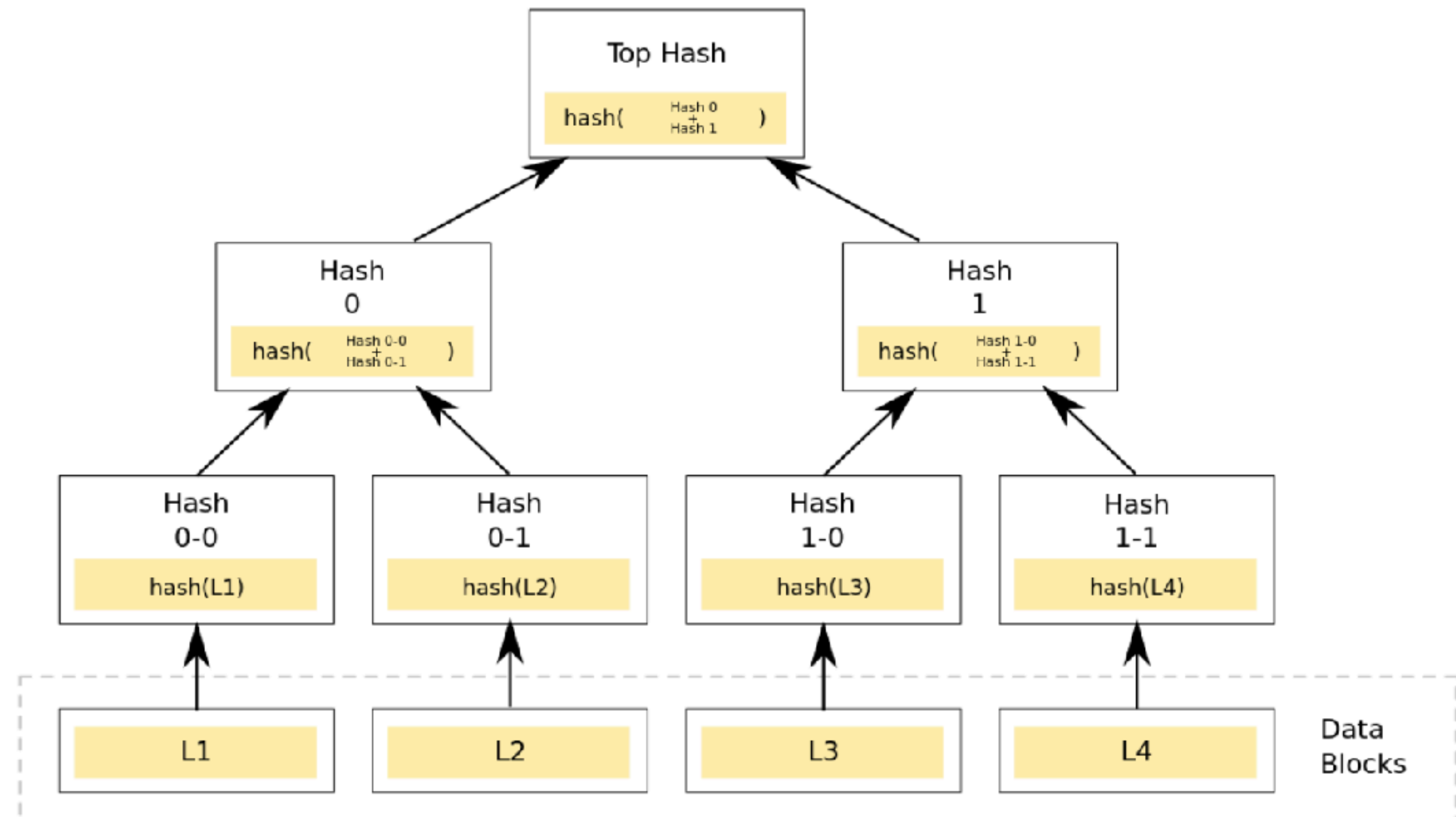
- A **pointer**: object that stores mem address
- A **hash pointer** additionally stores cryptographic hash.
- Tamper-evidence property



Merkle Trees

Moving towards efficiency

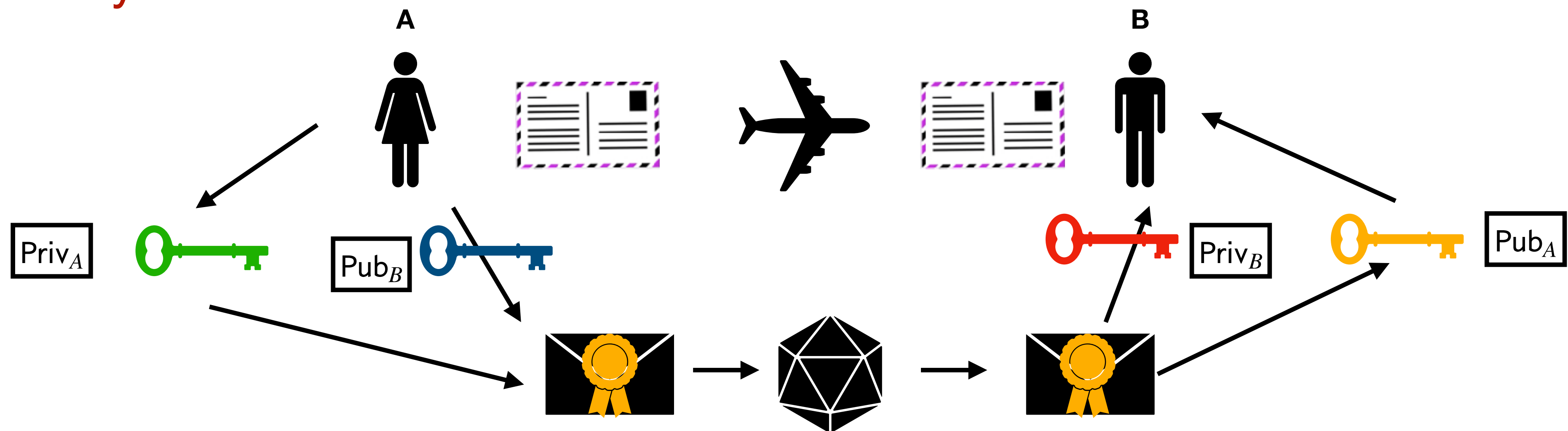
- Merkle tree is a binary tree of **hash pointers**
- Quick tamper-evidence check
- Concise proof of **membership**
- Proof of **non-membership** in log time and space



Digital Signatures

Identities and non-repudiation

- Properties:
 - Cannot be forged; Cannot be snipped off
- Public keys as identities — can make as many of them as one desires; need to be careful of **sybil attacks**





Digital Signatures

ECDSA common in Blockchain

- How are they different from MAC?
- Engineering Truth $\rightarrow$ Hash then sign!
- ECDSA setting:
 - Given a cyclic group generator G on an elliptic curve with curve order n
 - Private key sk = an integer $mod\ n$
 - Public key pk is $sk \cdot G$

Signing a Transaction

- Start with a message m ;
- Compute signature $\sigma = \text{Sign}(sk, m)$
- Broadcast (tx, σ, pk)
- Each node verifies: $\text{Verify}(pk, m, \sigma) = \text{True}$
-

MultiSignatures

Used In Bitcoins, Ethereum, Solana

- Against a transaction, one lists the public addresses of authorised signers
 - A threshold is set. Eg: 2-of-3 signers must sign for the transaction the go through
 - Enforces consent in the application layer
- Issues:
 - The **signers are fixed** and changing them would require a change in the script/contract
 - **Low privacy** (signer are all publicly known) and the **gas cost is high**: $O(n)$

Foundations of Threshold Signature Scheme (TSS)

- Relies on secure **Multi-Party Computation**
- **Actual workflow:**
 - **Distributed Key Generation:** All parties, say A, B, and C, run a protocol to generate parts of a private key x
 - A, B, and C hold secret shares of x : s_A, s_B, s_C respectively
 - **Signing:** Say A wants to send funds. A messages B and C. Alice computes a partial signature σ_A (using s_A) and so does B and C
 - Partial signatures are combined to obtain σ
 - A b'casts the txn with the combined signature σ

More on TSS

- Cheap ($O(1)$ gas cost); Takes place off-chain; high privacy
- TSS implementations — Schnorr's Signatures, ECDSA
- For distributed key generation:
 - Refer to Feldman's Verifiable Secret Shares algorithm



The System Models

- Network Synchrony
 - **Synchronous**: node actions are coordinated in rounds (achieved by a common clock)
 - Eg: All nodes bcast in round r , and in round $(r+1)$ all nodes receive the messages
 - **Asynchronous**: No common clock, no bound on n/w latency and drift
 - **Partially synchronous**: No common clock, but upper bound on message delivery
- Fault Model
 - **Crash failure**: Node can stop working abruptly and does not resume
 - **Byzantine failure**: Nodes act arbitrarily — can remain silent and modify messages



Distributed Consensus and Failure Models

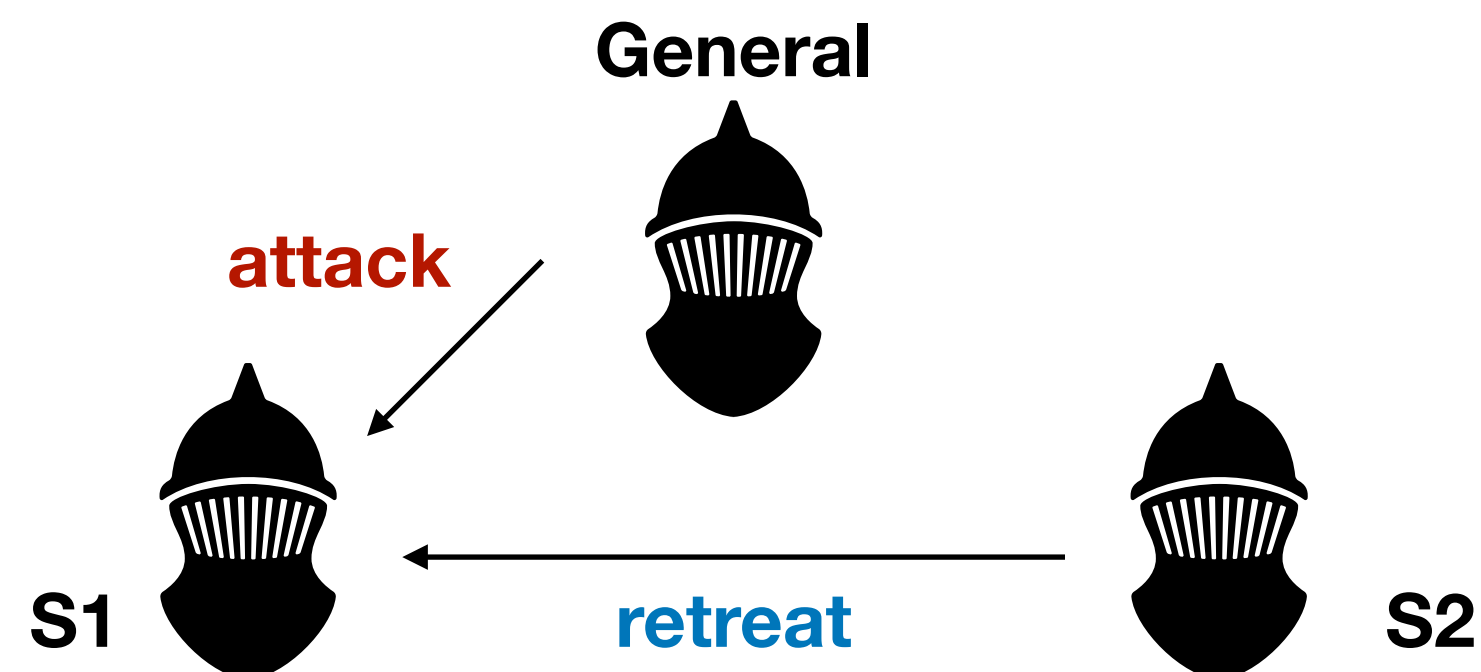
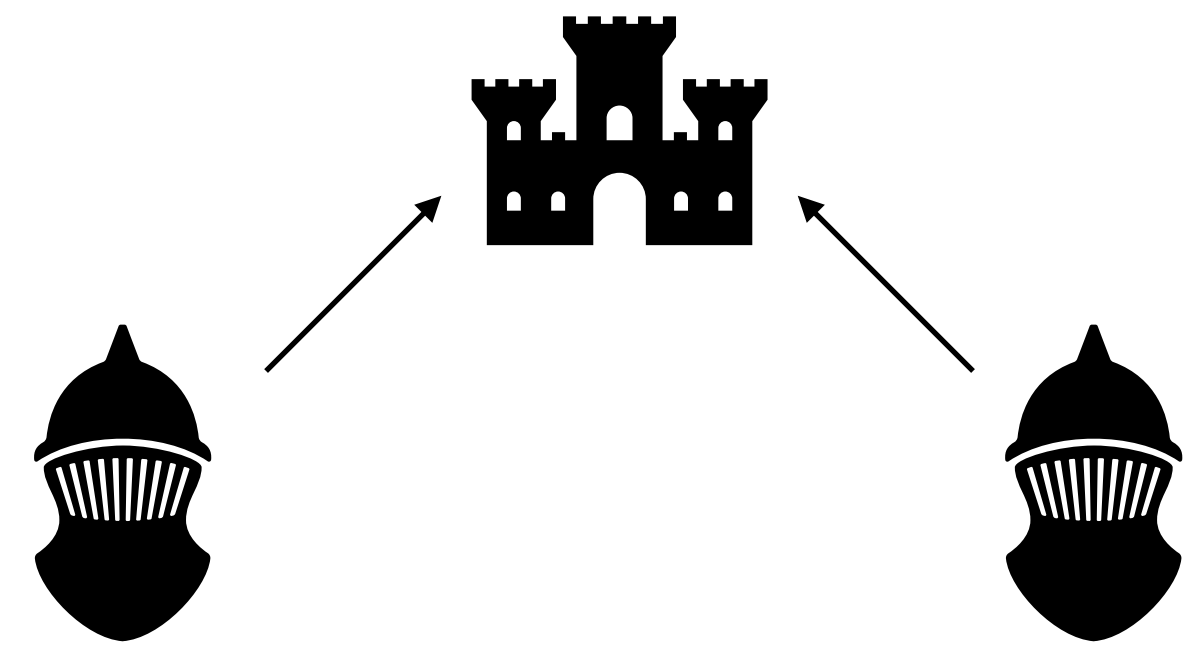
- The consensus problem:
 - **Agreement:** All honest nodes must agree on the same value
 - **Validity:** If all honest nodes have the same input value, then the agreed upon value is the same input value
 - **Termination:** Every honest node must eventually decide a value.

Failure Mode	Synchronous System	Asynchronous system
No failure	Agreement	Agreement
Crash failure	Agreement $f < n$	No agreement
Byzantine failure	Agreement $f \leq n/3$	No agreement

Table: Results on Agreement. n is the total number of processes and f is the number of failure-prone processes.

The Byzantine Agreement Problem

- A General commands soldiers
 - If all the soldiers attack then the castle can be won
 - If some retreat then the battle is lost
 - **Byzantine Agreement:** if the loyal (honest) soldiers have a majority agreement
 - **Impossibility Result:** Fisher-Lynch-Paterson, JACM'85
 - What should S1 do?
 - S1 can obey the commander
 - S1 and S2 must obey the same order





The Byzantine Agreement Problem

- For m faults, there must at least be $n = 3m + 1$ participants
 - Assumptions: No message loss; Reliable links; **Synchronous** systems
- Complexity: $O(n^m)$
- Unsigned messages: Faulty nodes can lie what others said
 - Use **digital signatures** — can't be forged
 - When node I passes signed message from node J, the recipient knows that node I did not lie about what node J said!